



JAVA BASICS

Practice Questions

15 Questions • Full Explanation • Dry Run

How to use this booklet

Each question has: the **concept used**, the full **code**, a **step-by-step dry run table** showing how variable values change, and the **final output**. Try solving each question yourself first, then check the dry run!

 Prepared by – **Akshit Sharma**

 akshbuilds.tech

 8510968871 /  akshubhardwaj231@gmail.com

1 Swap two numbers without using a third variable

💡 Concept

We can swap values using **arithmetic operators (+, -)** — add both numbers into one variable, then subtract to separate them back out in swapped order.

```
public class Swap {
    public static void main(String[] args) {
        int a = 5, b = 10;
        System.out.println("Before: a="+a+" b="+b);

        a = a + b;    // step 1
        b = a - b;    // step 2
        a = a - b;    // step 3

        System.out.println("After: a="+a+" b="+b);
    }
}
```

🔍 Dry Run (a=5, b=10)

Step	Operation	a	b
Start	—	5	10
1	a = a + b = 5+10	15	10
2	b = a - b = 15-10	15	5
3	a = a - b = 15-5	10	5

▶ Output

Before: a=5 b=10

After: a=10 b=5

2 Check if a number is even or odd

Concept

Use the **modulus operator (%)** which gives the remainder. If a number % 2 equals 0, it's even; otherwise, it's odd. Combined with an **if-else** statement.

```
public class EvenOdd {  
    public static void main(String[] args) {  
        int num = 7;  
        if (num % 2 == 0) {  
            System.out.println(num + " is Even");  
        } else {  
            System.out.println(num + " is Odd");  
        }  
    }  
}
```

Dry Run (num=7)

Step	Expression	Result
1	num % 2 → 7 % 2	1
2	is 1 == 0 ?	false
3	condition false → else block runs	"7 is Odd"

Output

7 is Odd

3 Find the largest of three numbers

Concept

Use **nested if-else** (or relational + logical operators) to compare all three numbers against each other and find the greatest one.

```
public class Largest {
    public static void main(String[] args) {
        int a = 12, b = 45, c = 30;
        if (a >= b && a >= c) {
            System.out.println("Largest: " + a);
        } else if (b >= a && b >= c) {
            System.out.println("Largest: " + b);
        } else {
            System.out.println("Largest: " + c);
        }
    }
}
```

Dry Run (a=12, b=45, c=30)

Step	Condition Checked	Result
1	a>=b && a>=c → 12>=45 && 12>=30	false
2	b>=a && b>=c → 45>=12 && 45>=30	true → runs this block

Output

Largest: 45

4 Print Fibonacci series up to n terms

Concept

Fibonacci series: each number is the **sum of the previous two** (0, 1, 1, 2, 3, 5...). Use a **for loop** with two tracking variables that update every iteration.

```
public class Fibonacci {  
    public static void main(String[] args) {  
        int n = 5, first = 0, second = 1;  
        for (int i = 1; i <= n; i++) {  
            System.out.print(first + " ");  
            int next = first + second;  
            first = second;  
            second = next;  
        }  
    }  
}
```

Dry Run (n=5)

i	Printed	next	first (after)	second (after)
1	0	0+1=1	1	1
2	1	1+1=2	1	2
3	1	1+2=3	2	3
4	2	2+3=5	3	5
5	3	3+5=8	5	8

Output

0 1 1 2 3

5 Check whether a number is Prime

🔗 Concept

A number is **prime** if it's divisible only by 1 and itself. Use a **for loop** from 2 to $n/2$ and check if any number divides it evenly ($\% == 0$).

```
public class Prime {
    public static void main(String[] args) {
        int num = 7;
        boolean isPrime = true;
        if (num <= 1) isPrime = false;
        for (int i = 2; i <= num/2; i++) {
            if (num % i == 0) {
                isPrime = false;
                break;
            }
        }
        System.out.println(isPrime ? num+" is Prime" : num+" is Not
Prime");
    }
}
```

🔗 Dry Run (num=7, loop runs i=2 to 3)

i	7 % i	== 0 ?	isPrime
2	1	false	true (unchanged)
3	1	false	true (unchanged)
loop ends (i=4 > num/2=3), isPrime stays true			

▶ Output

7 is Prime

6 Reverse a number using a while loop

🔗 Concept

Repeatedly extract the **last digit** using **% 10**, build the reversed number, then remove the last digit using **/ 10**. Continue while the number is not 0.

```
public class ReverseNum {
    public static void main(String[] args) {
        int num = 1234;
        int reversed = 0;
        while (num != 0) {
            int digit = num % 10;
            reversed = reversed * 10 + digit;
            num = num / 10;
        }
        System.out.println("Reversed: " + reversed);
    }
}
```

🔗 Dry Run (num=1234)

num	digit = num%10	reversed = rev*10+digit	num = num/10
1234	4	0*10+4 = 4	123
123	3	4*10+3 = 43	12
12	2	43*10+2 = 432	1
1	1	432*10+1 = 4321	0
num == 0 → loop stops			

▶ Output

Reversed: 4321

7 Find the sum of digits of a number

💡 Concept

Same digit-extraction technique as reversing — use **% 10** to get each digit and add it to a running total, then use **/ 10** to move to the next digit.

```
public class SumDigits {
    public static void main(String[] args) {
        int num = 456;
        int sum = 0;
        while (num != 0) {
            sum = sum + (num % 10);
            num = num / 10;
        }
        System.out.println("Sum of digits: " + sum);
    }
}
```

🔍 Dry Run (num=456)

num	num%10	sum = sum + digit	num = num/10
456	6	0+6 = 6	45
45	5	6+5 = 11	4
4	4	11+4 = 15	0
num == 0 → loop stops			

▶ Output

Sum of digits: 15

8 Find the factorial of a number using a for loop

Concept

Factorial of n ($n!$) = $n \times (n-1) \times (n-2) \times \dots \times 1$. Use a **for loop** that multiplies a result variable by each number from 1 up to n .

```
public class Factorial {  
    public static void main(String[] args) {  
        int n = 5;  
        long fact = 1;  
        for (int i = 1; i <= n; i++) {  
            fact = fact * i;  
        }  
        System.out.println(n + "! = " + fact);  
    }  
}
```

Dry Run (n=5)

i	$fact = fact * i$
1	$1 * 1 = 1$
2	$1 * 2 = 2$
3	$2 * 3 = 6$
4	$6 * 4 = 24$
5	$24 * 5 = 120$

Output

5! = 120

9 Print the multiplication table of a number

💡 Concept

Use a **for loop** that runs from 1 to 10, multiplying the given number by the loop counter each time to print each row of the table.

```
public class Table {
    public static void main(String[] args) {
        int num = 4;
        for (int i = 1; i <= 10; i++) {
            System.out.println(num + " x " + i + " = " + (num*i));
        }
    }
}
```

🔍 Dry Run (num=4, first 4 iterations)

i	num * i	Printed line
1	4	4 x 1 = 4
2	8	4 x 2 = 8
3	12	4 x 3 = 12
4	16	4 x 4 = 16
...continues till i=10		

▶ Output (partial)

```
4 x 1 = 4
4 x 2 = 8
4 x 3 = 12
... up to 4 x 10 = 40
```

10 Find the sum and average of array elements

💡 Concept

Loop through the array using `arr.length`, add every element to a sum variable, then divide the sum by the number of elements to get the average.

```
public class ArraySum {
    public static void main(String[] args) {
        int[] arr = {10, 20, 30, 40};
        int sum = 0;
        for (int i = 0; i < arr.length; i++) {
            sum = sum + arr[i];
        }
        double avg = (double) sum / arr.length;
        System.out.println("Sum: " + sum + ", Avg: " + avg);
    }
}
```

🔍 Dry Run (arr = {10,20,30,40})

i	arr[i]	sum = sum + arr[i]
0	10	0+10 = 10
1	20	10+20 = 30
2	30	30+30 = 60
3	40	60+40 = 100
avg = (double)100 / 4 = 25.0		

▶ Output

Sum: 100, Avg: 25.0

11 Find the largest element in an array

🔗 Concept

Assume the **first element** is the largest, then loop through the rest of the array comparing each element – update the "largest" variable whenever a bigger value is found.

```
public class MaxInArray {
    public static void main(String[] args) {
        int[] arr = {12, 45, 7, 89, 33};
        int max = arr[0];
        for (int i = 1; i < arr.length; i++) {
            if (arr[i] > max) {
                max = arr[i];
            }
        }
        System.out.println("Largest: " + max);
    }
}
```

🔗 Dry Run (arr = {12,45,7,89,33}, max starts = 12)

i	arr[i]	arr[i] > max ?	max (after)
1	45	45>12 true	45
2	7	7>45 false	45
3	89	89>45 true	89
4	33	33>89 false	89

▶ Output

Largest: 89

12 Check if a String is a Palindrome

🔗 Concept

A palindrome reads the same forwards and backwards (e.g. "madam"). Reverse the string manually using a loop, then compare it with the original using `.equals()`.

```
public class Palindrome {
    public static void main(String[] args) {
        String str = "madam";
        String rev = "";
        for (int i = str.length()-1; i >= 0; i--) {
            rev = rev + str.charAt(i);
        }
        if (str.equals(rev)) {
            System.out.println("Palindrome");
        } else {
            System.out.println("Not a Palindrome");
        }
    }
}
```

🔗 Dry Run (str = "madam", length=5)

i	str.charAt(i)	rev (after)
4	'm'	"m"
3	'a'	"ma"
2	'd'	"mad"
1	'a'	"mada"
0	'm'	"madam"
rev="madam" equals str="madam" → true		

▶ Output

Palindrome

13 Count the number of vowels in a string

Concept

Loop through each character using `charAt(i)`, and check if it matches a vowel (a, e, i, o, u – both cases) using logical OR (`||`). Increment a counter for each match.

```
public class CountVowels {
    public static void main(String[] args) {
        String str = "Akshit";
        int count = 0;
        for (int i = 0; i < str.length(); i++) {
            char ch = Character.toLowerCase(str.charAt(i));
            if (ch=='a' || ch=='e' || ch=='i' || ch=='o' || ch=='u') {
                count++;
            }
        }
        System.out.println("Vowels: " + count);
    }
}
```

Dry Run (str = "Akshit")

i	ch (lowercase)	is vowel?	count
0	a	yes	1
1	k	no	1
2	s	no	1
3	h	no	1
4	i	yes	2
5	t	no	2

Output

Vowels: 2

14 Build a simple calculator using switch-case

Concept

Use a **switch statement** to choose which arithmetic operation to perform based on an operator character. Each **case** handles one operator, with **break** to stop fall-through.

```
public class Calculator {
    public static void main(String[] args) {
        int a = 10, b = 5;
        char op = '*';
        int result;
        switch(op) {
            case '+': result = a+b; break;
            case '-': result = a-b; break;
            case '*': result = a*b; break;
            case '/': result = a/b; break;
            default: result = 0;
        }
        System.out.println("Result: " + result);
    }
}
```

Dry Run (a=10, b=5, op='*')

Step	Action
1	op matches case '*'
2	result = a*b = 10*5 = 50
3	break → exits switch

Output

Result: 50

15 Write a method to check if a year is a Leap Year

Concept

A year is a leap year if it's **divisible by 4**, but NOT by 100 — unless it's also divisible by 400. This uses a **method with a return value** and logical operators (&&, ||).

```
public class LeapYear {
    static boolean isLeap(int year) {
        return (year % 4 == 0 && year % 100 != 0) || (year % 400 ==
0);
    }
    public static void main(String[] args) {
        int year = 2024;
        boolean result = isLeap(year);    // method call
        System.out.println(year + " Leap Year? " + result);
    }
}
```

Dry Run (year=2024)

Expression	Value
year % 4 == 0 → 2024%4	0 → true
year % 100 != 0 → 2024%100	24 → true
(true && true)	true
true (year%400==0)	true (short-circuit, 2nd part skipped)

Output

2024 Leap Year? true

🎉 Well Done! 🎉

You've completed all 15 practice questions.

Try re-writing each program yourself without looking, then dry-run it on paper!

Happy Coding! ☕

Prepared by – Akshit Sharma

🌐 akshbuilds.tech

📞 8510968871 / ✉ akshubhardwaj231@gmail.com